

3rdparty_net_snmp enhancement

Created by  Vincent Tung (董建隆), last updated on 2026/03/23 • 27 minute read

 Scroll page details

Net-SNMP Refactoring Plan: 3rdparty_net_snmp

Document Info

Field	Value
Package	3rdparty_net_snmp (Net-SNMP 5.8 + Moxa Extensions)
Date	2026-03-23
Authors	Platform Engineering, Moxa Inc.
Status	Draft - Internal Review
Version	1.0

1. Executive Summary

Critical Performance Findings

The current 3rdpartynetsnmp package has six identified performance bottlenecks. Three are classified **CRITICAL** and cause directly measurable user-visible degradation. Every SNMP SET operation blocks snmpd for up to **4.2 seconds** due to hardcoded sleep() calls. Every SNMP GET on a cold cache spawns multiple processes via fork/exec, each taking **10-150 ms** on the ARM Cortex-A9. The C code is compiled with **-O0** (no optimisation) and the Rust binaries use **opt-level = "s"** (size-optimised, not speed-optimised), leaving significant CPU performance on the table.

Tier Summary

Tier	Label	Items	Effort	Risk	Expected Gain
Tier 1	Quick Wins	Compiler flags, cache TTL	1-2 days	Low	20-40% throughput increase
Tier 2	Structural Fixes	Remove sleep() delays, parallel URI fetch, pipe IPC	1-2 weeks	Medium	SET 4.2s -> ~200ms; GET -50%
Tier 3	Architecture Change	In-process background cache (Option A)	3-4 weeks	Medium-High	GET <1ms (cache hit), fork/exec eliminated

Key Metrics: Current vs. Proposed

Metric	Current	Proposed (Tier 1+2)	Proposed (Tier 3)
SNMP GET latency (cold cache)	10-150 ms	5-80 ms	<1 ms (cache hit)
SNMP GET latency (warm cache)	10-150 ms	5-80 ms	<1 ms
SNMP SET latency	4.2 s	~200 ms	~200 ms
fork/exec per GET	2-4	2-4	0 (background)
File I/O ops per GET	3-6	1-2	0 (memory)
C compiler optimisation	-O0	-O3 -fllto	-O3 -fllto
Rust optimisation	opt-level="s"	opt-level=3	opt-level=3
Cache TTL (config data)	800 ms	10 s	10 s (tiered)

2. Package Structure & Moxa Proprietary Modifications

Net-SNMP 5.8 is a widely-used open-source SNMP agent toolkit. Moxa has extended it with a proprietary MIB handler subsystem called **iesautomibs** that translates SNMP OID requests into HTTP URI calls against the ISS (Internal Settings Server) REST API running on 127.0.0.1:80. The MIB handler is entirely Moxa-written and lives in net-snmp-5.8/agent/mibgroup/ies-auto-mibs/.

Moxa-Specific Files and Their Purpose

File	Purpose
iesautomibs.c	Main MIB handler: registers OIDs, dispatches GET/SET, manages file cache, drives fork/exec for Rust tools
moxasnmphandle_util.c	SET utility functions: URI config/status/command helpers, contains the blocking sleep() calls
moxasnmpconst.h	Central constants: buffer sizes, timeouts, URI path macros, IPC socket path
moxasnmphandle_event.c	Background event handler thread (pthread): UDS socket server for save/exit actions
iesautomibsetup*.c	90+ database handler files, one per subsystem (port, VLAN, RSTP, PoE, QoS, LLDP, etc.)
moxasnmphandleentryset_value.c	Entry-level SET dispatch

<code>moxasnmphandleentryget_value.c</code>	Entry-level GET dispatch
<code>moxasnmphandleentrydependency_check.c</code>	Pre-SET dependency validation
<code>moxasnmppconstla.h</code> , <code>moxasnmppconstvlan.h</code> , etc.	Subsystem-specific constants

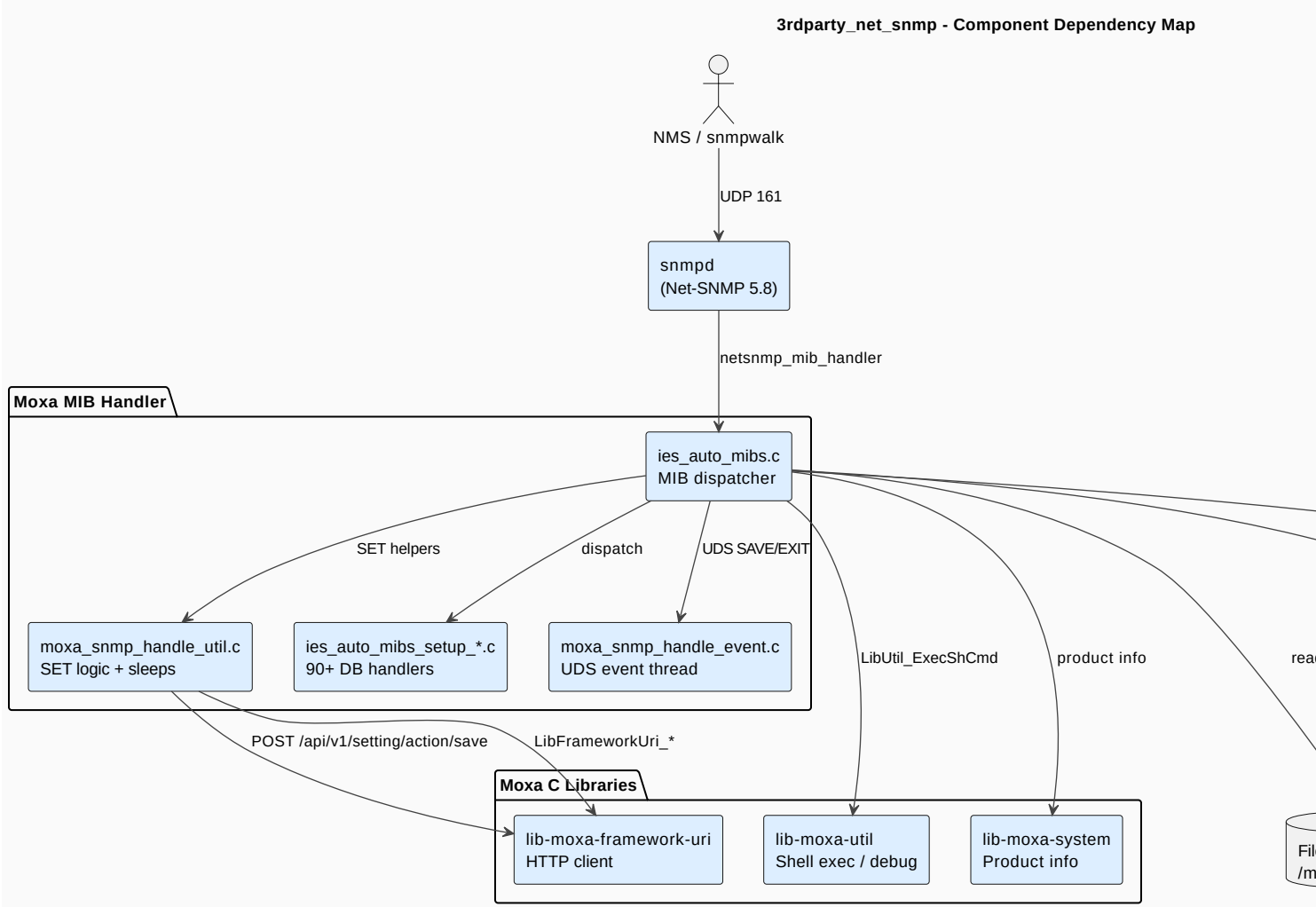
Rust Binaries (cross-compiled for armv7-unknown-linux-gnueabihf)

Binary	Purpose
<code>generateurisnmp_table</code>	Fetches URI data from ISS and writes SNMP table cache files
<code>generateisssnmpmergetable</code>	Merges ISS remap tables for port-indexed MIB tables

Moxa Libraries Linked

Library	Role
<code>lib-moxa-framework-uri</code> (<code>libmoxaframeworkuri</code>)	HTTP client for ISS REST API communication
<code>lib-moxa-util</code> (<code>libmoxa_util</code>)	Shell command execution (<code>LibUtil_ExecShCmd</code>), debug tracing
<code>lib-moxa-system</code> (<code>libmoxa_system</code>)	Product ID / hardware info
<code>libmoxarust_iss-custom</code>	Rust crate providing ISS API access to Rust binaries

Dependency Diagram



3. Current Architecture -- How It Works

3.1 Data Flow Narrative

When an NMS issues an SNMP GET, snmpd receives the UDP packet on port 161 and dispatches it to the registered MIB handler for that OID subtree. For Moxa-managed OIDs, this calls `moxasnmphandleentry()` in `iesauto_mibs.c`.

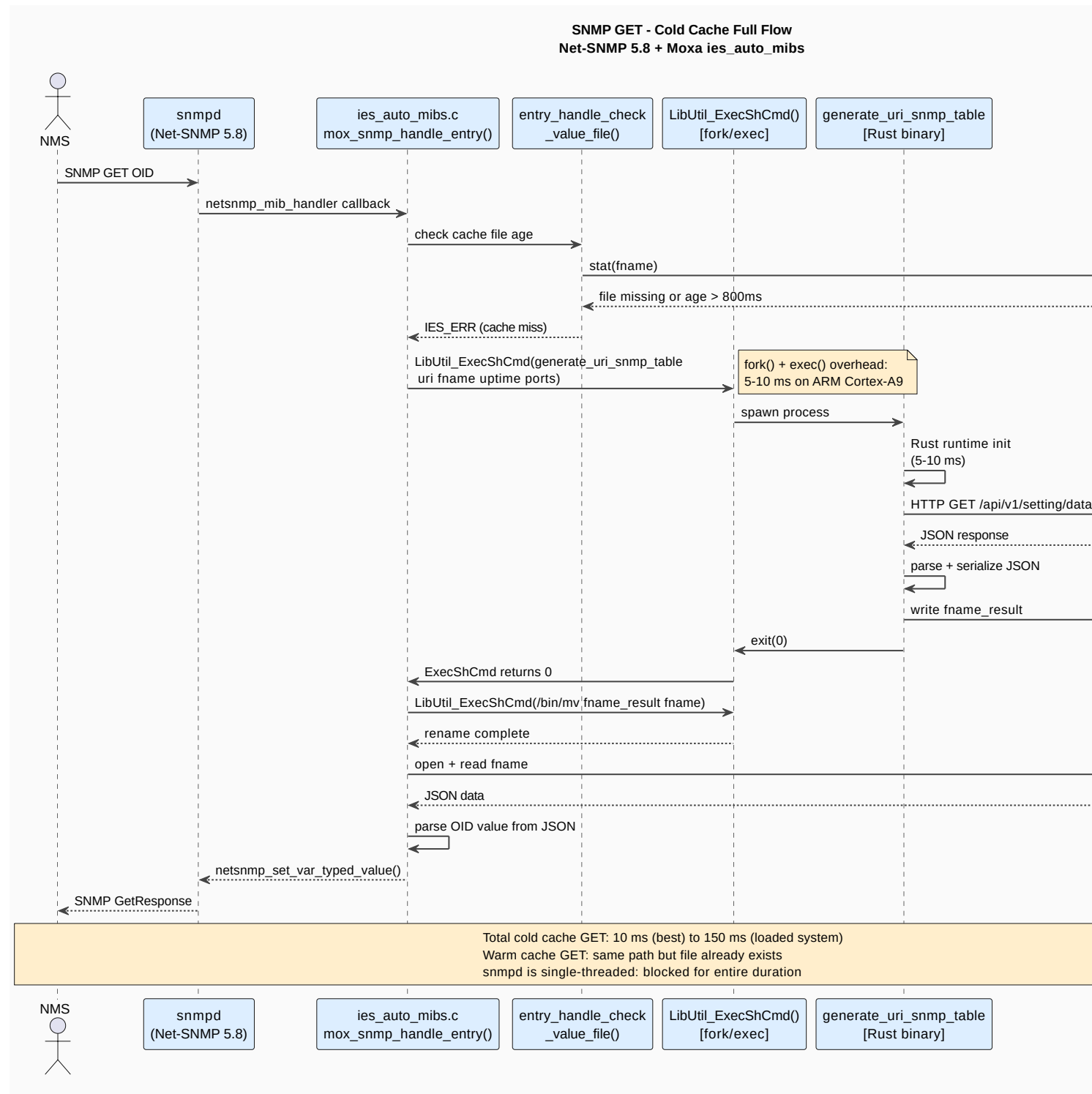
The handler checks whether a valid file cache entry exists for the requested URI. Cache validity is determined by two `CLOCKMONOTONIC` timestamps stored in `gSnmpTblSnapshot`: `timeTemp` (time since cache file was created) and `timeLastUpdate` (time since last access within an active walk). The timeout constants are `MOXASNMPMAXTABLEENTRYTIMEOUT = 800,000,000 ns (800 ms)` and `MOXASNMPMAXTABLEENTRYLASTACCESSTIME = 200,000,000 ns (200 ms)`.

On a **cold cache** (file missing or expired), the handler calls `entryhandlegenerateissvaluefile()` which invokes `LibUtilExecShCmd()` to fork/exec one of the Rust binaries. The Rust binary starts up, initialises its own HTTP client (`libmoxarustiss-custom`), makes one or more HTTP GET requests to the ISS REST API at `/api/v1/setting/data/` or `/api/v1/status/`, serialises the result to a temporary file in `/moxa/run/automibs/`, then exits. Control returns to `snmpd`, which reads the file and parses the JSON to extract the OID value.

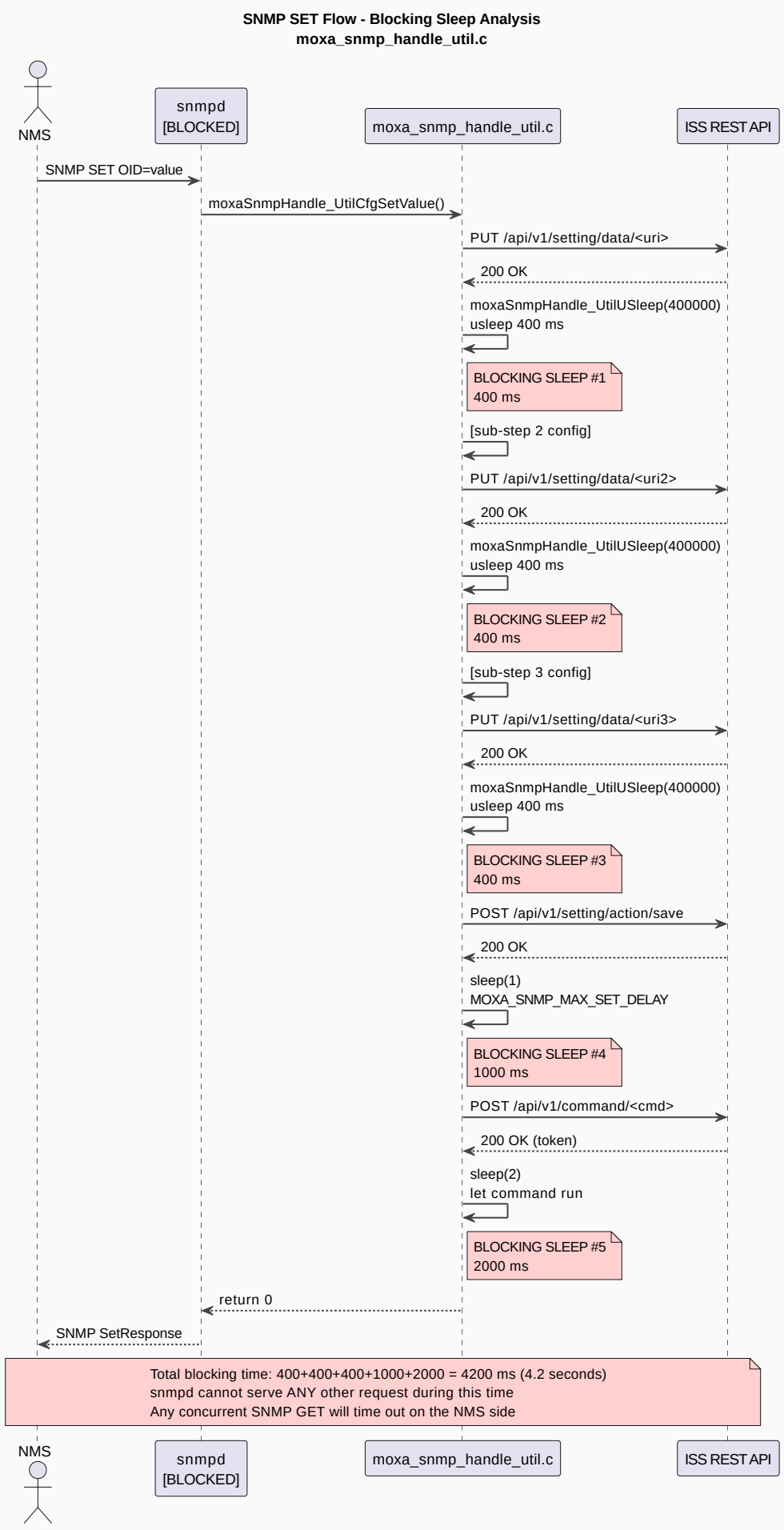
On a **warm cache**, `entryhandlecheckvaluefile()` confirms the file is fresh and the handler reads directly from the existing file without forking. However, since 800 ms is very short and many NMS tools poll faster than this, cold-cache paths are hit frequently.

For SNMP SET operations, `moxasnmphandleutil.c` submits the change via `LibFrameworkUriPostUri()` to the ISS REST API, then calls `sleep(MOXASNMPMAXSETDELAY)` (1 second) after the save action, and `moxaSnmpHandleUtilSleep(MOXASNMPMAXCFG_DELAY)` (400,000 us = 400 ms) after each sub-step. A command-type SET additionally calls `sleep(2)` after the command POST. For a complete SET transaction involving config + 3 sub-steps + command, total blocking time reaches **4.2 seconds**.

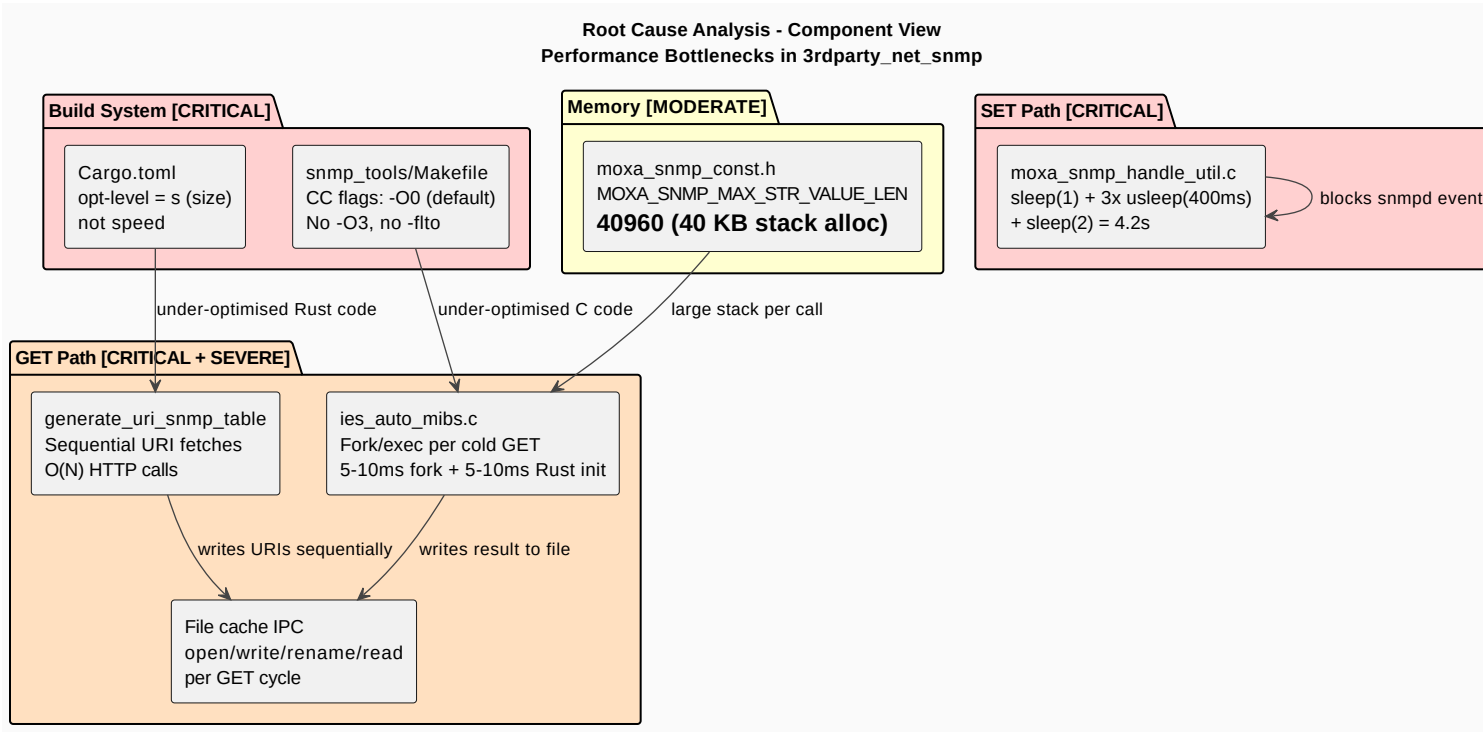
3.2 GET Flow: SNMP GET Cold Cache (Full Sequence)



3.3 SET Flow: Hardcoded Blocking Delays



3.4 Root Cause Component Diagram



LDFLAGS += -flto

Proposed Fix: Cargo.toml
Proposed: Cargo.toml - speed-optimised release

```
[profile.release]
strip = true
opt-level = 3      # Optimise for speed
lto = true
codegen-units = 1
```

Quick Win - Tier 1

Compiler flag changes require no code modification. Rebuild and test. Expected impact: 20-40% reduction in Rust binary execution time. On a warm-cache GET path, this directly reduces snmpd handler latency.

4.2 Hardcoded Blocking Delays -- CRITICAL

Three locations in moxasnmpandleutil.c call moxaSnmpHandleUtilUSleep(MOXASNMPMAXCFGDELAY) (400,000 us = 400 ms each), and one calls sleep(MOXASNMPMAXSETDELAY) (1 second). In moxasnmpandleevent.c / moxasnmpandleutil.c, the command trigger path calls sleep(2). All of these run **synchronously on the snmpd main thread**, blocking the entire SNMP agent.

Code showing the sleep pattern in moxasnmpandle_util.c

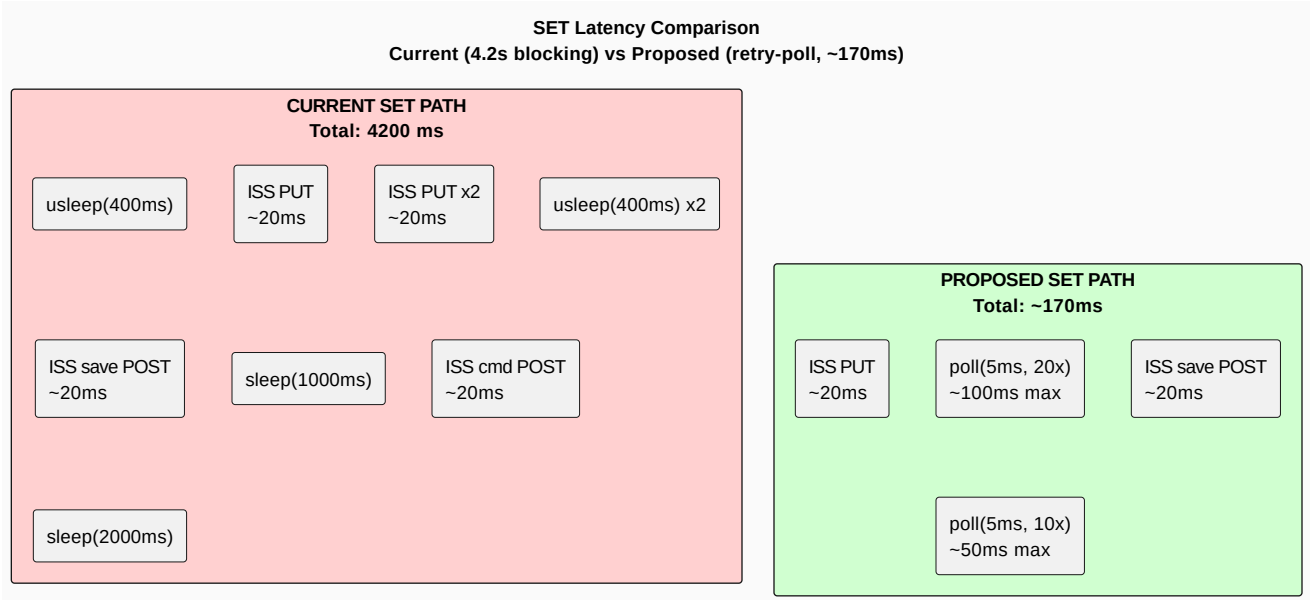
```
moxa_snmp_handle_util.c - blocking sleep locations

/* After each URI config sub-step (called up to 3 times): */
if (LibFrameworkUri_PostUri(gUriHandle, uriDataPath, security_token,
                             jsonVal, &jsonValSetRet) != 0) {
    goto config_end;
}
moxaSnmpHandle_UtilUSleep(MOXA_SNMP_MAX_CFG_DELAY); /* 400,000 us = 400 ms BLOCKING */

/* After save action (moxaSnmpHandle_UtilCfgSetRunningToStartupConfigAndSleep): */
if (LibFrameworkUri_PostUri(gUriHandle, uriPath, security_token,
                             NULL, &jsonValRet) != 0) {
    goto config_end;
}
sleep(MOXA_SNMP_MAX_SET_DELAY); /* 1 second BLOCKING */

/* After command trigger (moxaSnmpHandle_UtilCfgTriggerURICmd): */
if (LibFrameworkUri_PostUri(gUriHandle, uriDataPath, security_token,
                             jsonVal, &jsonValCmdResult) != 0) {
    goto config_end;
}
sleep(2); /* let command run - 2 seconds BLOCKING */
```

Timing Diagram: Current vs Proposed SET latency



Proposed Fix: Replace sleep() with retry-poll

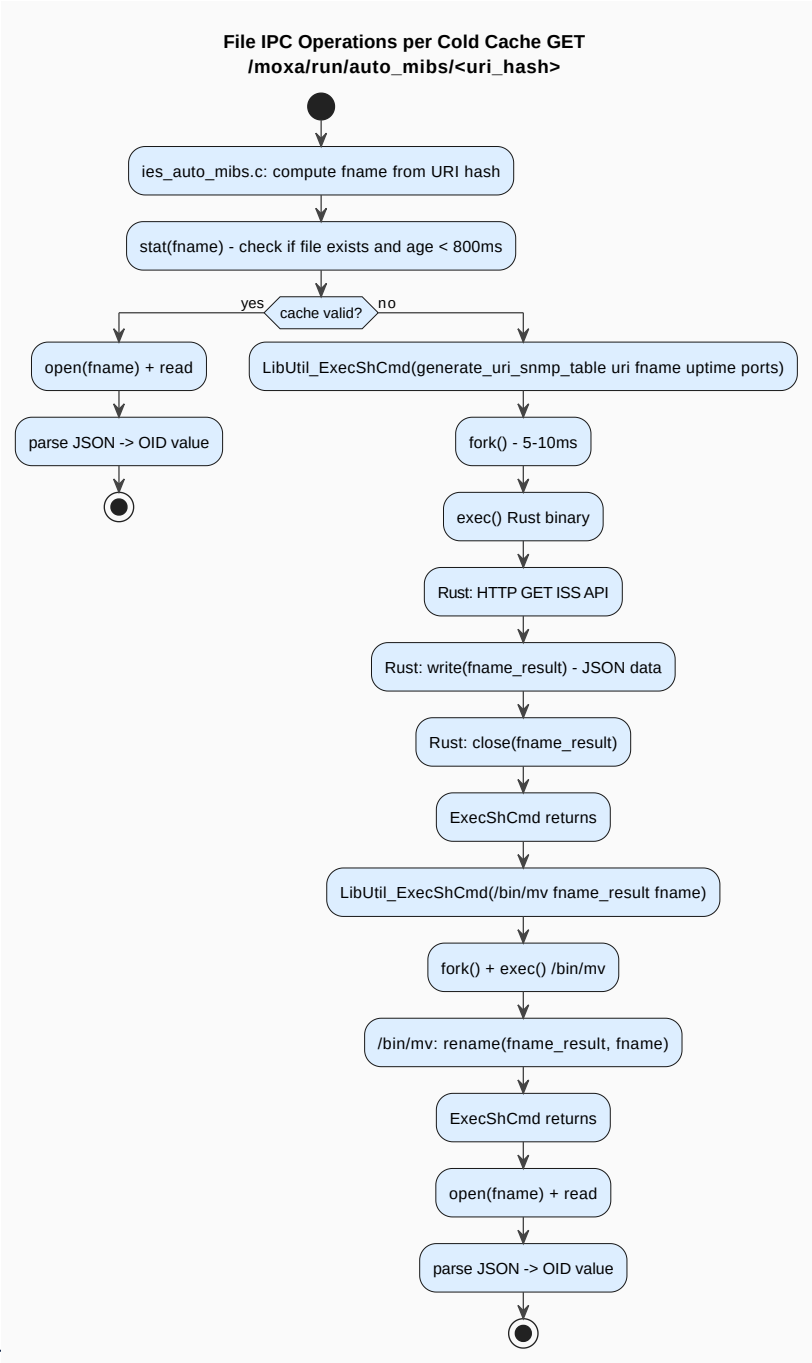
Proposed: retry-poll pattern replaces blocking sleep

```
/* Replace moxaSnmpHandle_UtilUSleep(MOXA_SNMP_MAX_CFG_DELAY) with: */
static int poll_iss_ready(struct LibFrameworkUri_Handle *h,
                          const char *status_uri,
                          const char *token,
                          int max_retries, int poll_ms)
{
    for (int i = 0; i < max_retries; i++) {
        JSON_Value *jv = NULL;
        if (0 == LibFrameworkUri_GetUri(h, status_uri, token, &jv)) {
            int ready = iss_status_is_ready(jv);
            if (jv) json_value_free(jv);
            if (ready) return 0;
        }
        struct timeval tv = { .tv_sec = 0, .tv_usec = poll_ms * 1000 };
    }
}
```

```
select(0, NULL, NULL, NULL, &tv);
}
return -1; /* timed out */
}
/* Usage: poll up to 20 times x 5ms = 100ms maximum wait */
poll_iss_ready(gUriHandle, MOXA_SNMP_URI_STATUS_PATH("system/ready"),
security_token, 20, 5);
```

4.3 File-Based IPC -- CRITICAL

Every GET on a cold cache results in the Rust binary writing its result to a temporary file, which is then renamed into place by snmpd. This involves: `open()` for write, multiple `write()` calls, `fsync()` or `close()`, `rename()` (which on Linux is atomic but still a syscall), and `open()` + `read()` from snmpd. On the ARM Cortex-A9 with an eMMC flash storage device, each file operation takes 0.5-2 ms. For a table walk with 32 entries, this is 100-400 ms of file I/O alone.



File operation flowchart per GET

Current constants (moxasnmplib.h)

Current: fixed 800ms TTL for all data types

```
#define MOXA_SNMP_MAX_TABLE_ENTRY_TIMEOUT      (800 * 1000 * 1000) /* 800ms in nanoseconds */
#define MOXA_SNMP_MAX_TABLE_ENTRY_LAST_ACCESS_TIME (200 * 1000 * 1000) /* 200ms */
```

Proposed fix: tiered TTL based on data volatility

Proposed: tiered TTL constants

```
/* Tiered cache TTL based on data category */
#define MOXA_SNMP_TTL_CONFIG_NS      (10000 * 1000 * 1000LL) /* 10s - config/setting data */
#define MOXA_SNMP_TTL_STATUS_NS      (800 * 1000 * 1000LL) /* 800ms - status/counters */
#define MOXA_SNMP_TTL_EVENT_NS       (200 * 1000 * 1000LL) /* 200ms - event/alarm data */
```



```
/* Derive TTL from URI prefix */
static inline int64_t moxa_snmp_get_ttl_ns(const char *uri) {
    if (strncmp(uri, "/api/v1/setting/data/", 21) == 0) return MOXA_SNMP_TTL_CONFIG_NS;
    if (strncmp(uri, "/api/v1/status/", 15) == 0) return MOXA_SNMP_TTL_STATUS_NS;
    return MOXA_SNMP_TTL_EVENT_NS;
}
```

4.4 Fork/Exec Per Query -- SEVERE

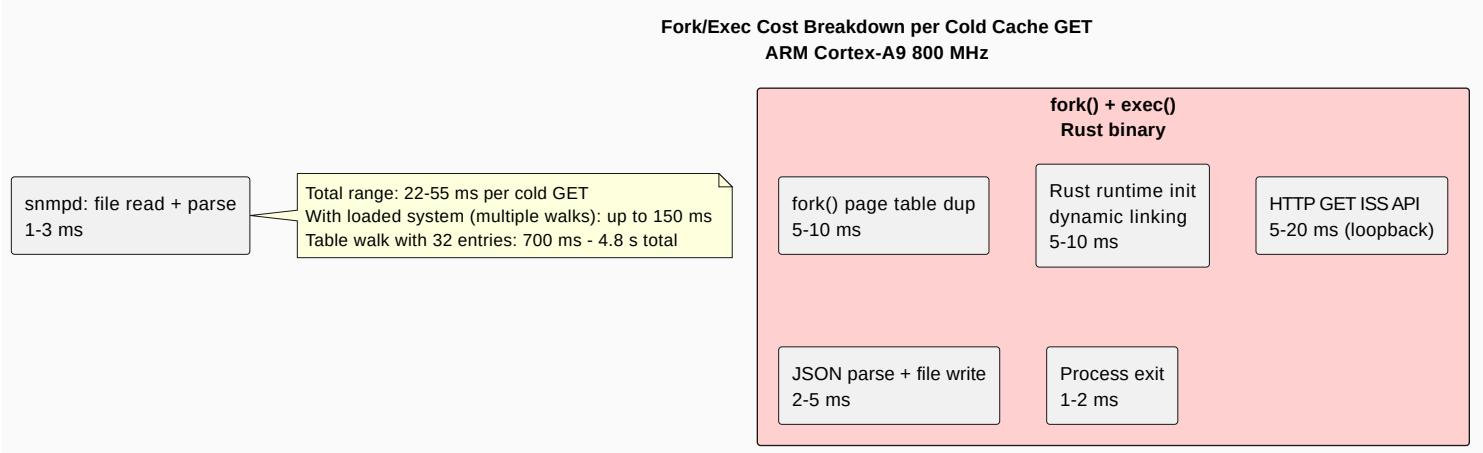
iesautomibs.c calls LibUtil_ExecShCmd() to fork and exec the Rust binary on **every cold cache GET**. On an ARM Cortex-A9 at 800 MHz, fork() takes approximately 5-10 ms (due to page table duplication even with copy-on-write). The Rust binary then requires another 5-10 ms to initialise its runtime, link shared libraries, and open the HTTP client. Additionally, /bin/mv is fork/exec'd separately to rename the result file. For a full SNMP table walk over 32 entries with a 800 ms cache TTL, snmpd can trigger 32 fork/exec cycles within a single walk.

Code showing the fork/exec pattern

ies_auto_mibs.c - fork/exec per GET

```
/* entry_handle_generate_iss_value_file() - called on every cold cache GET */
if (0 != LibUtil_ExecShCmd(NULL, NULL, 0,
    "/sbin/generate_uri_snmp_table "
    " %s %s %s_result %ld %d",
    entry->uri, fname, fname, info.uptime, gMaxPortInDevice))
{
    LibUtil_DbgTrace(kDbgFlgIssUriSNMP, "Error in %s line %d\n", __func__, __LINE__);
    return mox_snmp_flask_handle_result(uri_handle, json_val, ret);
}
if (0 != LibUtil_ExecShCmd(NULL, NULL, 0,
    "/bin/mv %s_result %s", fname, fname))
{
    LibUtil_DbgTrace(kDbgFlgIssUriSNMP, "Error in %s line %d\n", __func__, __LINE__);
}
```

Cost breakdown per cold cache GET



Fix Option 1: Pipe IPC (replace file IPC)

Proposed: pipe IPC eliminates rename and reduces file ops

```
/* Use popen() instead of LibUtil_ExecShCmd() + file write/read */
FILE *pipe = popen("/sbin/generate_uri_snmp_table --pipe "
    "--uri " uri " --uptime " uptime, "r");

if (pipe) {
    /* read JSON directly from stdout pipe - no temp file */
    fread(json_buf, 1, sizeof(json_buf), pipe);
    pclose(pipe);
}
```

Fix Option 2: cdylib (eliminate fork/exec entirely for Option A)

Proposed: build as cdylib for in-process loading

```
# In generate_uri_snmp_table/Cargo.toml
[lib]
crate-type = ["cdylib"]
name = "generate_uri_snmp_table"
```

snmpd dlopen()s the .so and calls directly - no fork/exec

4.5 Sequential URI Fetches -- SEVERE

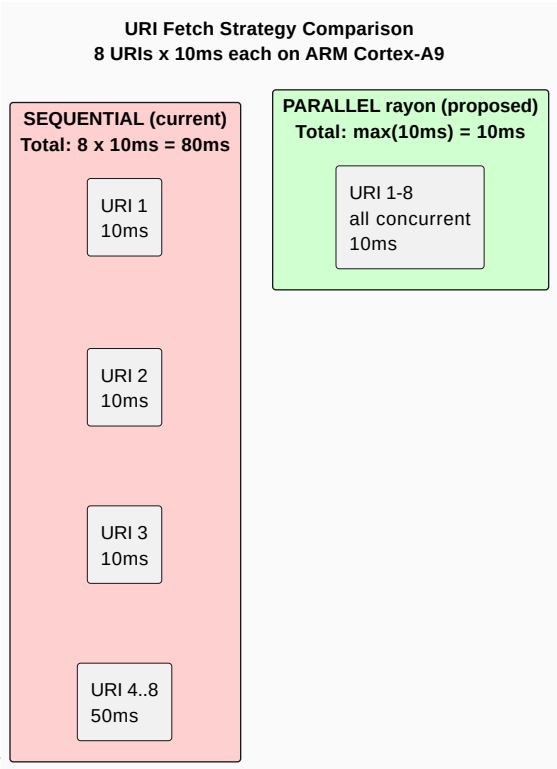
The Rust binary generateurisnmp_table processes URI lists sequentially. For complex MIB tables that require data from multiple ISS endpoints (e.g., port tables that fetch per-port config + status + counters), each HTTP request to ISS completes before the next begins. On loopback (127.0.0.1), each HTTP GET to the Flask-based ISS takes approximately 5-20 ms depending on ISS load. A table requiring 8 URI fetches takes 40-160 ms in the Rust binary alone, in addition to the fork/exec overhead.

Code showing sequential URI loop (representative pattern)

Current: sequential URI fetch in generate_uri_snmp_table

```
// Simplified representation of current sequential pattern:
for uri in &uri_list {
    let response = iss_client.get(uri)?; // blocks until complete
```

```
let data = response.json::<Value>()?;
results.push(data);
}
// Total time: sum of all individual HTTP round-trips
```



Timing comparison: sequential vs parallel

Proposed fix: rayon parallel iterator
Proposed: parallel URI fetch with rayon

```
use rayon::prelude::*;

// Replace sequential loop with parallel:
let results: Vec<Value> = uri_list
    .par_iter() // rayon parallel iterator
    .filter_map(|uri| {
        iss_client_tl() // thread-local ISS client
        .get(uri)
        .ok()
        .and_then(|r| r.json::<Value>().ok())
    })
    .collect();
// Total time: max(individual times) instead of sum
// Add to Cargo.toml: rayon = "1.8"
```

4.6 Memory Allocation in Hot Path -- MODERATE

moxasnmplib.h defines MOXASNMPLIB_MAXSTRVALUE_LEN = 40960 (40 KB). This constant is used as a stack-allocated buffer in the GET handler hot path. On the ARM Cortex-A9 with its small L1 data cache (typically 32 KB), a 40 KB stack frame evicts the entire working set from L1 on every function call. This is a moderate issue that amplifies the cost of the other bottlenecks.

Code showing the 40 KB stack allocation
moxa_snmp_const.h - oversized buffer constant

```
#define MOXA_SNMP_MAX_STR_VALUE_LEN (40960) /* origin is 8192 - comment from Moxa devs */
/* Used as: char val_str[MOXA_SNMP_MAX_STR_VALUE_LEN]; on the stack */
```

Proposed fix: thread-local static buffer
Proposed: thread-local static - single allocation, stays in cache

```
/* Replace stack VLA with thread-local static buffer */
static __thread char val_str[MOXA_SNMP_MAX_STR_VALUE_LEN];
/* Allocated once per thread, survives across calls, no stack pressure */
/* snmpd is single-threaded for handler callbacks - no lock needed */
```

Severity Map

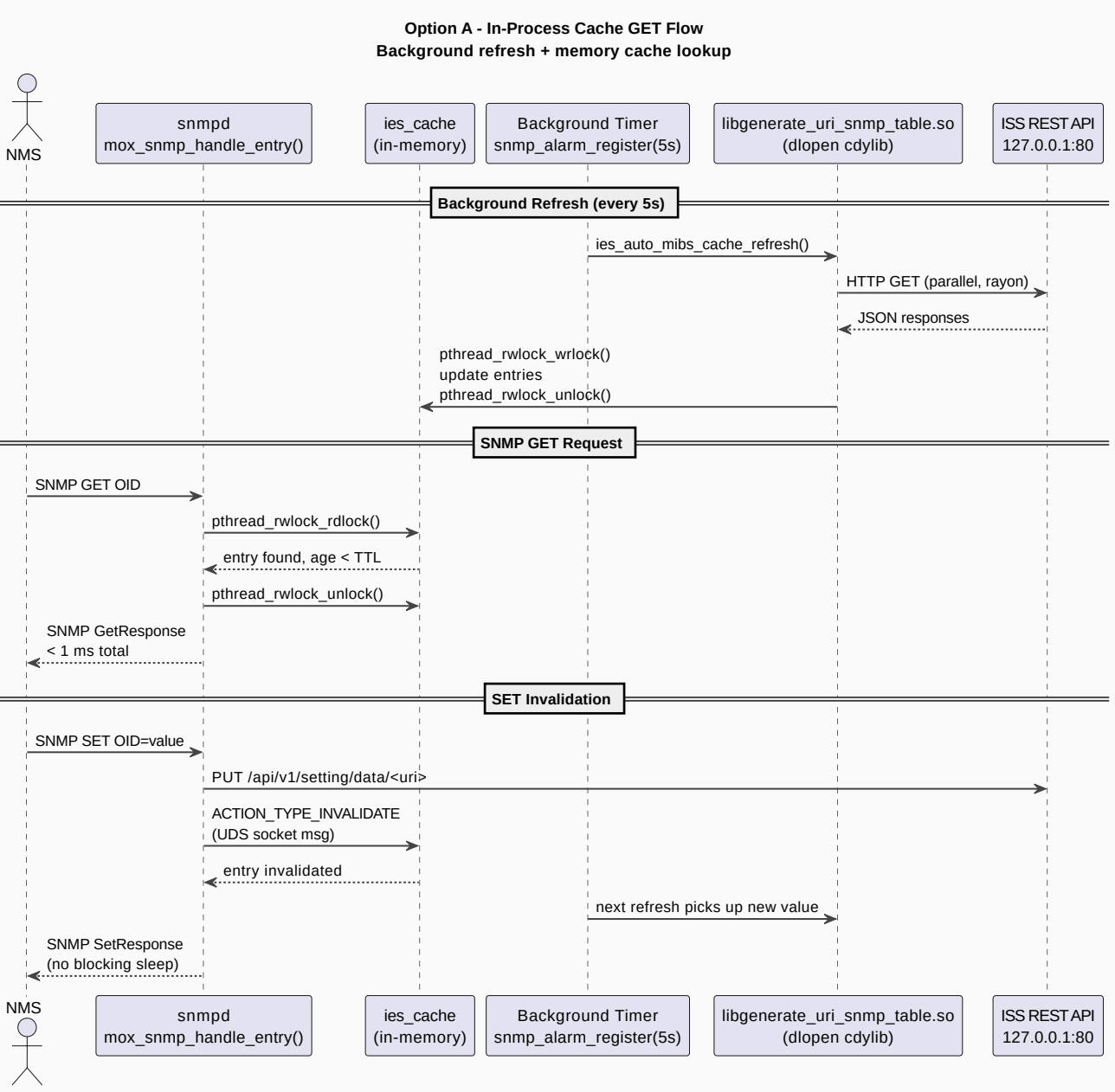


6. Proposed Architecture Options

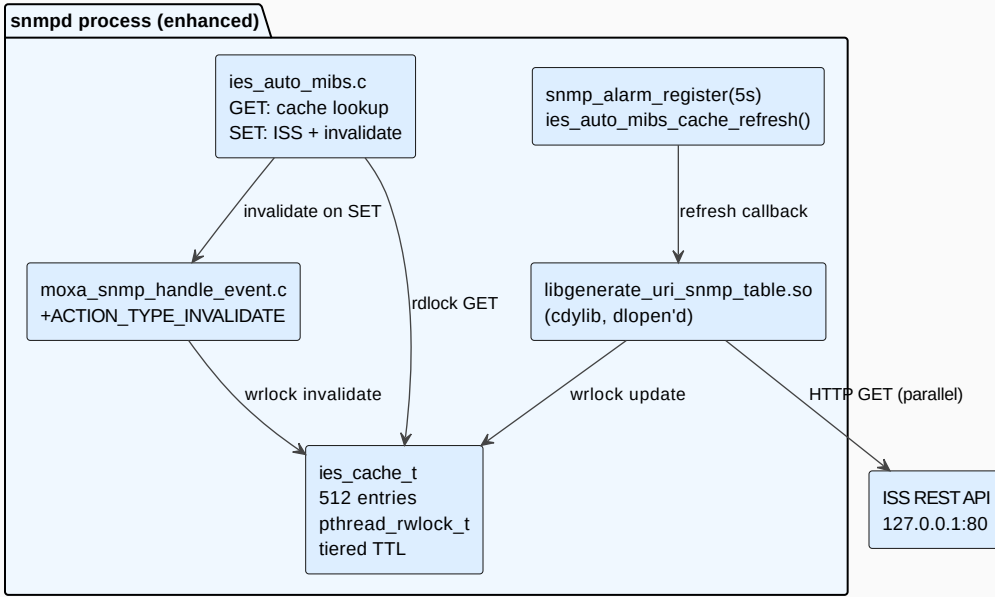
Option A eliminates the fork/exec pattern entirely by maintaining an in-memory cache inside the `snmpd` process. A background timer (using Net-SNMP's `snmp_alarm_register()` API) periodically refreshes cache entries by calling the Rust code as a dynamically-loaded library (`cdylib`) rather than a separate process. SNMP GET requests are served from the in-memory cache with a `pthread_rwlock_t` read lock, returning in under 1 ms. Cache misses on first access trigger a synchronous fetch, but subsequent GETs are always cache hits until the TTL expires.

The event handler thread (already present in `noxasnmphandleevent.c`) is extended with an `ACTIONTYPE_INVALIDATE` message so that SET operations can immediately invalidate affected cache entries rather than waiting for TTL expiry.

GET Flow: Background Timer + Cache Lookup



Option A - In-Process Cache Component Map



Component Map: Option A

Key Code Changes

Option A: `snmp_alarm_register` in `init_ies_auto_mibs()`

```
/* In init_ies_auto_mibs() - register background refresh timer */
snmp_alarm_register(5, /* period: 5 seconds */
    SA_REPEAT, /* repeat indefinitely */
    ies_auto_mibs_cache_refresh,
    NULL); /* callback arg */
```

Option A: `ies_cache_t` data structure

```
#define IES_CACHE_MAX_ENTRIES 512
#define IES_CACHE_MAX_URI 400
#define IES_CACHE_MAX_DATA (4 * 1024 * 1024) /* 4 MB */

typedef struct {
    char uri[IES_CACHE_MAX_URI];
    char *data; /* heap-allocated JSON */
    size_t data_len;
    struct timespec last_refresh;
    int64_t ttl_ns; /* tiered: 10s/800ms/200ms */
    int valid;
} ies_cache_entry_t;
```

```
typedef struct {
    ies_cache_entry_t entries[IES_CACHE_MAX_ENTRIES];
    pthread_rwlock_t lock;
    int count;
} ies_cache_t;
```

```
static ies_cache_t g_ies_cache = {
    .lock = PTHREAD_RWLOCK_INITIALIZER
};
```

Option A: new GET path (replaces fork/exec)

```
/* In mox_snmp_handle_entry() for GET - replaces entry_handle_generate_iss_value_file() */
static ies_handle_t ies_cache_get(const char *uri, char **out_data, size_t *out_len)
{
    struct timespec now;
    clock_gettime(CLOCK_MONOTONIC, &now);

    pthread_rwlock_rdlock(&g_ies_cache.lock);
    for (int i = 0; i < g_ies_cache.count; i++) {
        ies_cache_entry_t *e = &g_ies_cache.entries[i];
        if (!e->valid || strcmp(e->uri, uri) != 0) continue;

        int64_t age_ns = (int64_t)(now.tv_sec - e->last_refresh.tv_sec) * 1000000000LL
            + (int64_t)(now.tv_nsec - e->last_refresh.tv_nsec);
        if (age_ns < e->ttl_ns) {
            *out_data = e->data;
            *out_len = e->data_len;
            pthread_rwlock_unlock(&g_ies_cache.lock);
            return IES_NOERR; /* cache hit - < 1ms */
        }
    }
    pthread_rwlock_unlock(&g_ies_cache.lock);
    return IES_ERR; /* cache miss - trigger background refresh */
}
```

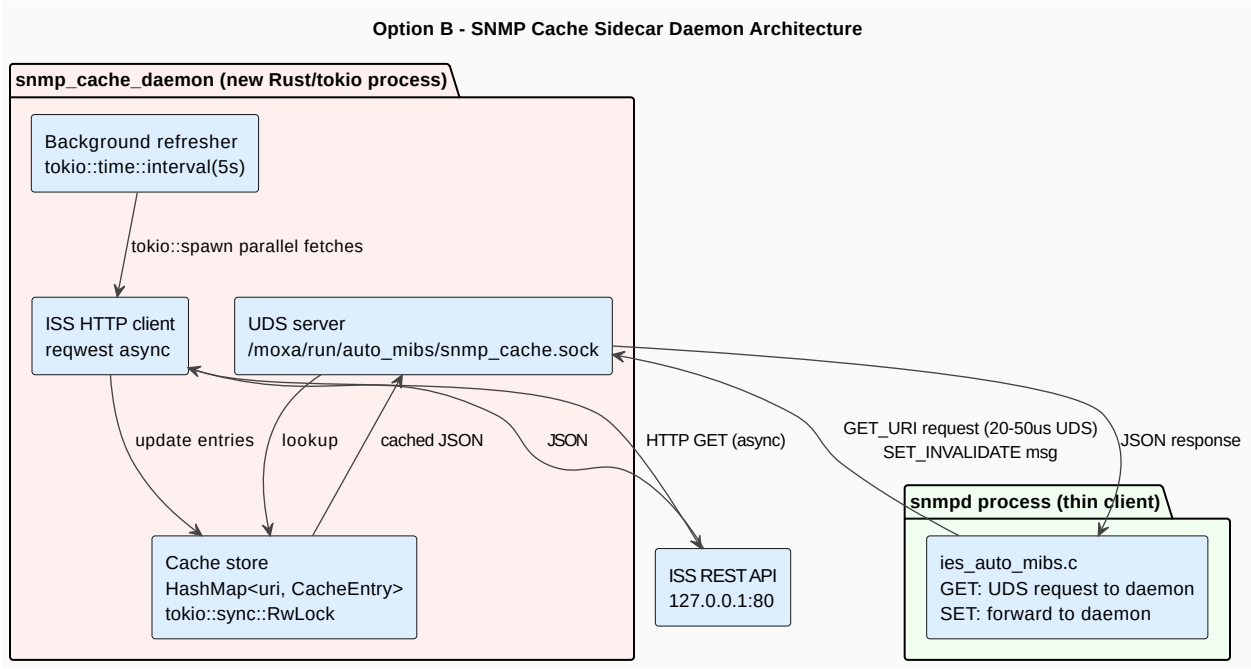
Pros/Cons: Option A

Aspect	Assessment
GET latency	<1 ms (cache hit), eliminates fork/exec entirely
SET latency	~200 ms (retry-poll replaces sleep)
Memory usage	~4-16 MB additional heap for cache data
Complexity	Medium: requires cdylib build, dlopen, alarm registration
Risk	Medium: rwlock contention if alarm fires during snmpd walk
snmpd restartability	Cache lost on restart; first-access cold-start
ISS compatibility	No ISS changes required
Effort	3-4 weeks

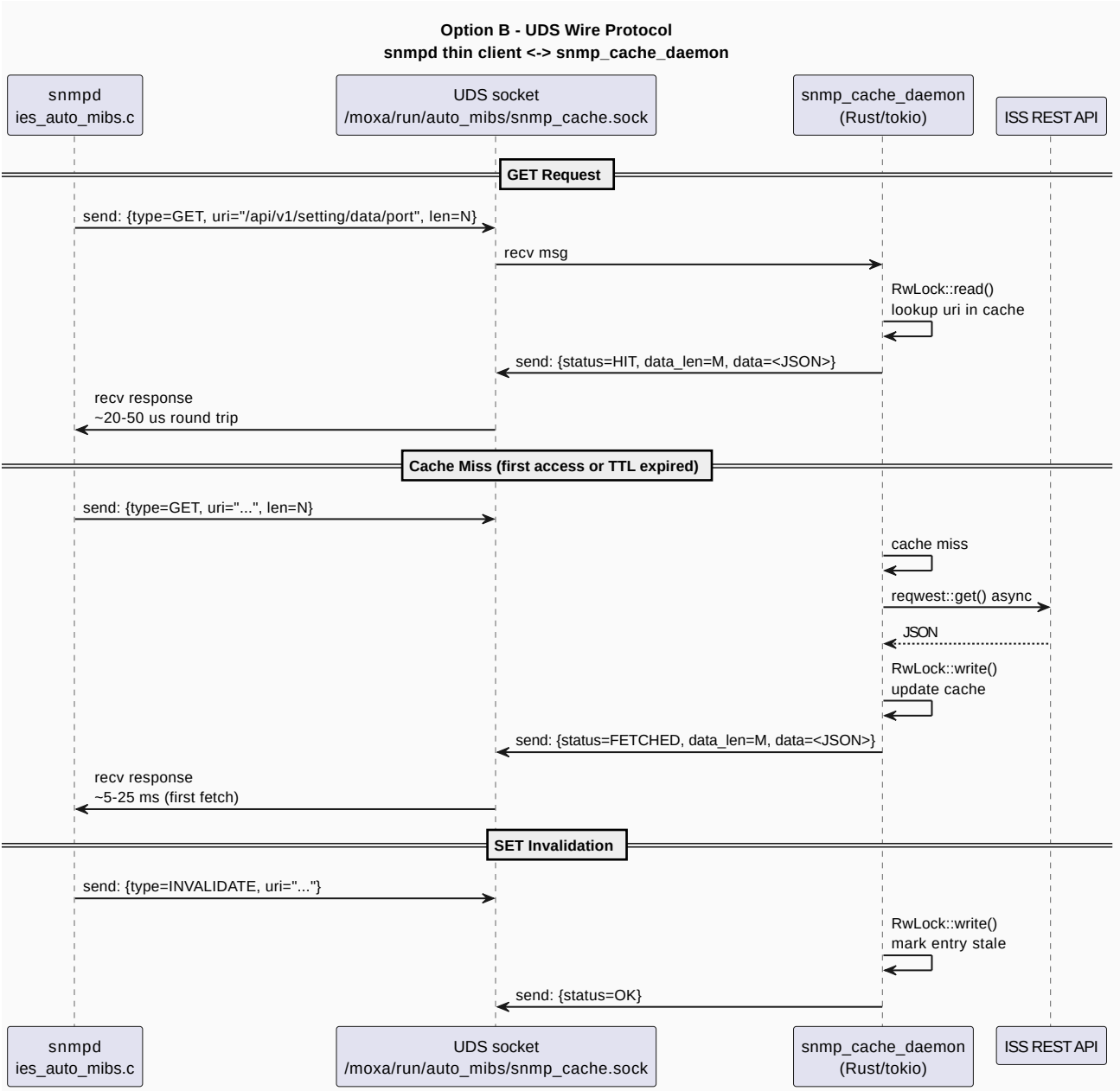
6.2 Option B -- SNMP Cache Sidecar Daemon

Option B moves the cache and ISS communication entirely out of snmpd into a standalone Rust daemon (snmpcachedaemon) built with tokio for async I/O. snmpd communicates with the daemon over a Unix Domain Socket (UDS). This completely decouples the SNMP agent lifecycle from the ISS polling lifecycle. The daemon can refresh data proactively at configurable intervals without holding any snmpd locks.

Component Architecture: Option B



UDS Wire Protocol Sequence



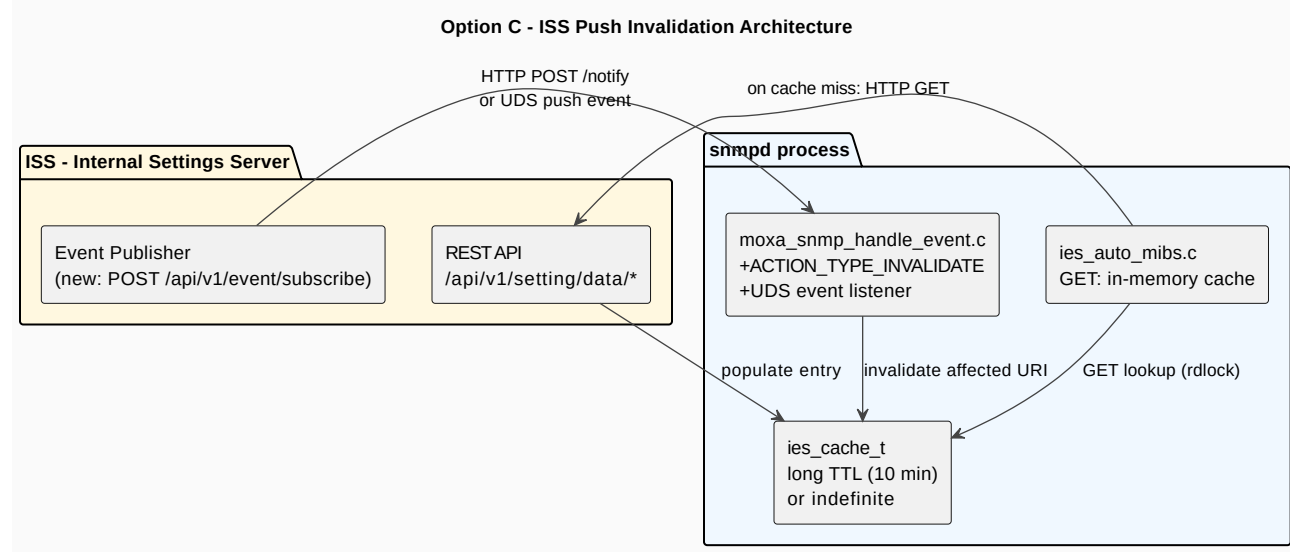
Pros/Cons: Option B

Aspect	Assessment
GET latency	~50 us UDS round-trip (cache hit); 5-25 ms (cache miss)
SET latency	~200 ms (no snmpd blocking)
Isolation	Excellent: daemon crash does not crash snmpd
Memory	Separate process; cache in daemon heap
Complexity	High: new Rust tokio daemon, startup ordering, systemd unit
Risk	Low for snmpd; medium for daemon reliability
Effort	5-6 weeks

6.3 Option C -- ISS Push Invalidation

Option C extends the existing event architecture so that ISS proactively notifies snmpd when configuration data changes, eliminating the need for TTL-based polling. This requires adding an event subscription mechanism to ISS and extending the `moxasnmphandleevent.c` protocol with `ACTIONTYPE_INVALIDATE`. The cache can then hold data indefinitely and only refresh when ISS signals a change, dramatically reducing both ISS load and GET latency.

Event-Driven Component Diagram



Extended Protocol: ACTIONTYPEINVALIDATE

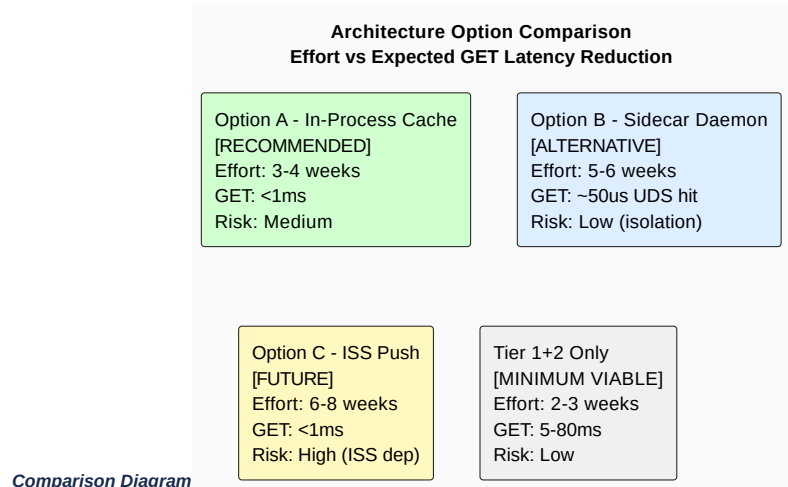
Option C: extend s_action_msg for invalidation

```
struct s_action_msg {
#define ACTION_TYPE_SAVE      1
#define ACTION_TYPE_EXIT     2
#define ACTION_TYPE_INVALIDATE 3 /* NEW: ISS push notification */
    uint32_t type;
    union {
        struct s_type_save      type_save;
        struct {
            char uri[MOXA_SNMP_MAX_URI_LENGTH]; /* URI prefix to invalidate */
        } type_invalidate;
    };
};
/* ISS calls: send ACTION_TYPE_INVALIDATE via UDS socket SOCK_PATH */
```

Pros/Cons: Option C

Aspect	Assessment
GET latency	<1 ms (cache hit, always valid)
SET latency	~200 ms (no snmpd blocking)
Cache staleness	Near-zero: push-invalidated on every change
ISS changes	Required: event subscription API in ISS
Complexity	Highest: requires ISS cooperation, event bus design
Risk	High: depends on ISS event delivery reliability
Effort	6-8 weeks (includes ISS changes)

7. Architecture Comparison



Comparison Diagram

Full Metrics Comparison Table

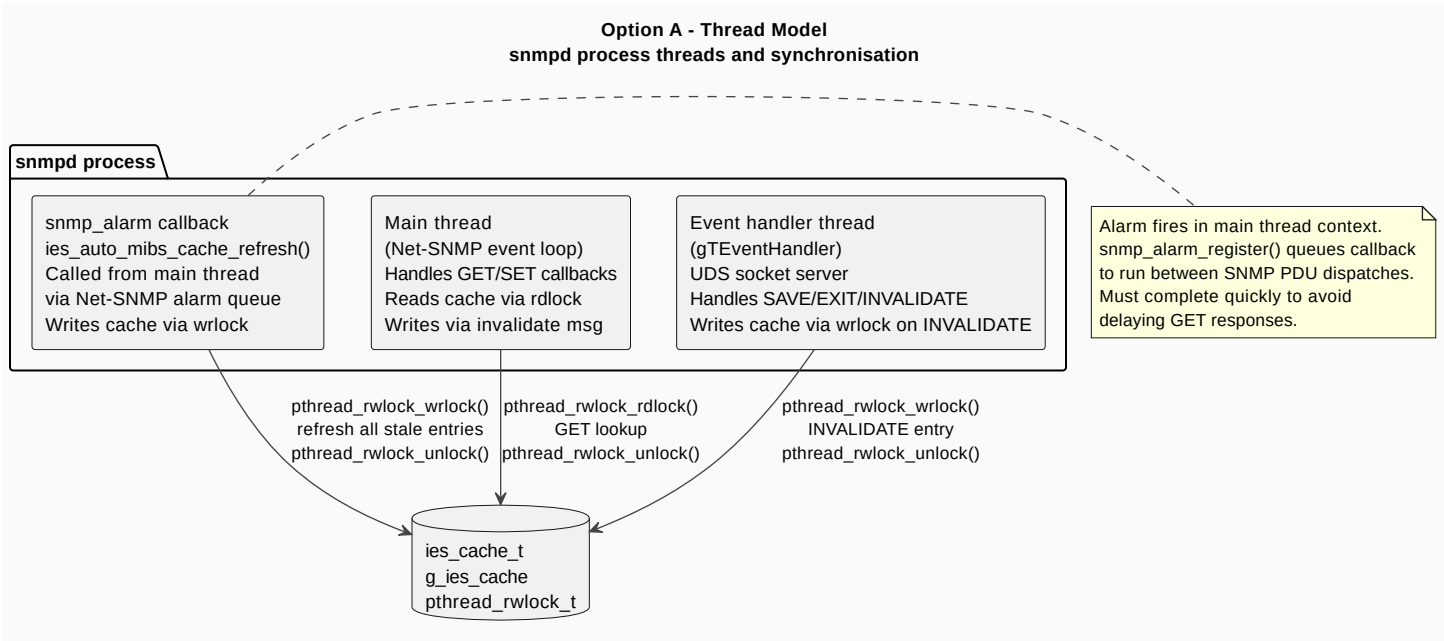
Metric	Current	Tier 1+2 Only	Option A	Option B	Option C
GET latency (cache hit)	10-150 ms	5-80 ms	<1 ms	~50 us	<1 ms
GET latency (cold)	10-150 ms	5-80 ms	5-25 ms	5-25 ms	5-25 ms
SET latency	4.2 s	~200 ms	~200 ms	~200 ms	~200 ms
File I/O per GET	3-6 ops	1-2 ops	0 (memory)	0 (memory)	0 (memory)
Fork/exec per walk	2-4	2-4	0	0	0
Cache staleness	800 ms	800ms / 10s	5 s (timer)	5 s (timer)	Near-zero
snmpd coupling	High	High	Medium	Low	Medium
ISS changes required	No	No	No	No	Yes
New processes	None	None	None	snmpcachedaemon	None
Implementation effort	-	2-3 weeks	3-4 weeks	5-6 weeks	6-8 weeks
Risk level	-	Low	Medium	Low	High

Recommendation

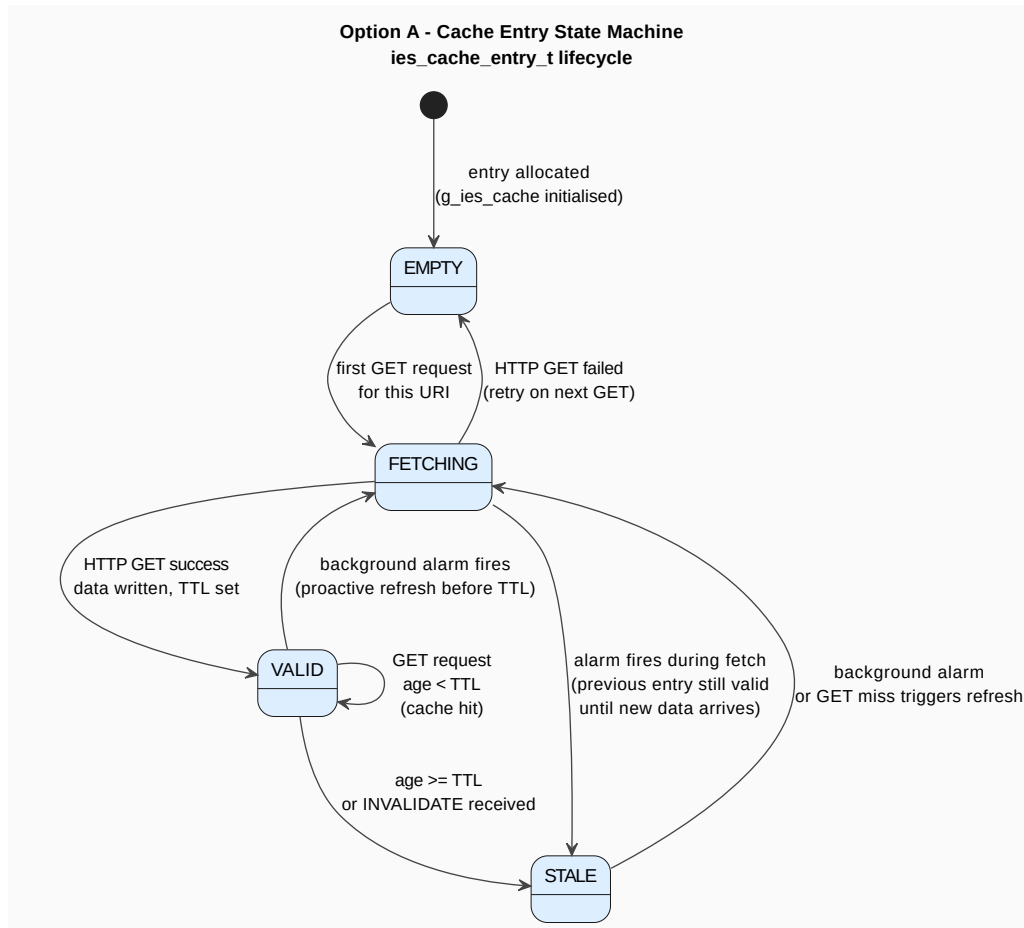
Implement **Tier 1 and Tier 2** fixes first (2-3 weeks, low risk): compiler flags, sleep() replacement with retry-poll, tiered TTL, rayon parallel fetch. This eliminates the 4.2s SET blocking and reduces GET latency significantly. Then proceed with **Option A** (in-process background cache) as the target architecture. Option B is viable if snmpd process isolation becomes a requirement. Option C should be deferred until ISS event infrastructure is available.

8. Detailed Design: Option A

8.1 Thread Model



8.2 Cache Entry State Machine



8.3 Cache Data Structure

Option A: Full ies_cache_t data structure

```

/*
 * In-process SNMP data cache for ies_auto_mibs
 * File: net-snmp-5.8/agent/mibgroup/ies-auto-mibs/ies_auto_mibs_cache.h
 */

#include <pthread.h>
#include <time.h>
#include <stdint.h>
#include <stddef.h>

#define IES_CACHE_MAX_ENTRIES    512
#define IES_CACHE_MAX_URI        400
#define IES_CACHE_MAX_DATA_LEN   (4 * 1024 * 1024) /* 4 MB per entry max */

/* Tiered TTL: derived from URI prefix */
#define IES_CACHE_TTL_CONFIG_NS   (10000LL * 1000000LL) /* 10 seconds */
#define IES_CACHE_TTL_STATUS_NS   ( 800LL * 1000000LL) /* 800 ns */
#define IES_CACHE_TTL_EVENT_NS    ( 200LL * 1000000LL) /* 200 ns */

typedef enum {
    IES_CACHE_EMPTY    = 0,
    IES_CACHE_FETCHING = 1,
    IES_CACHE_VALID     = 2,
    IES_CACHE_STALE     = 3,
} ies_cache_state_t;

typedef struct {
    char      uri[IES_CACHE_MAX_URI];
    char      *data; /* heap-allocated, NULL if empty */
    size_t    data_len;
    struct timespec last_refresh; /* CLOCK_MONOTONIC */
    int64_t    ttl_ns; /* assigned from URI prefix */
    ies_cache_state_t state;
} ies_cache_entry_t;

typedef struct {
    ies_cache_entry_t entries[IES_CACHE_MAX_ENTRIES];
    pthread_rwlock_t lock;
    int count; /* number of registered URIs */
}

```

```

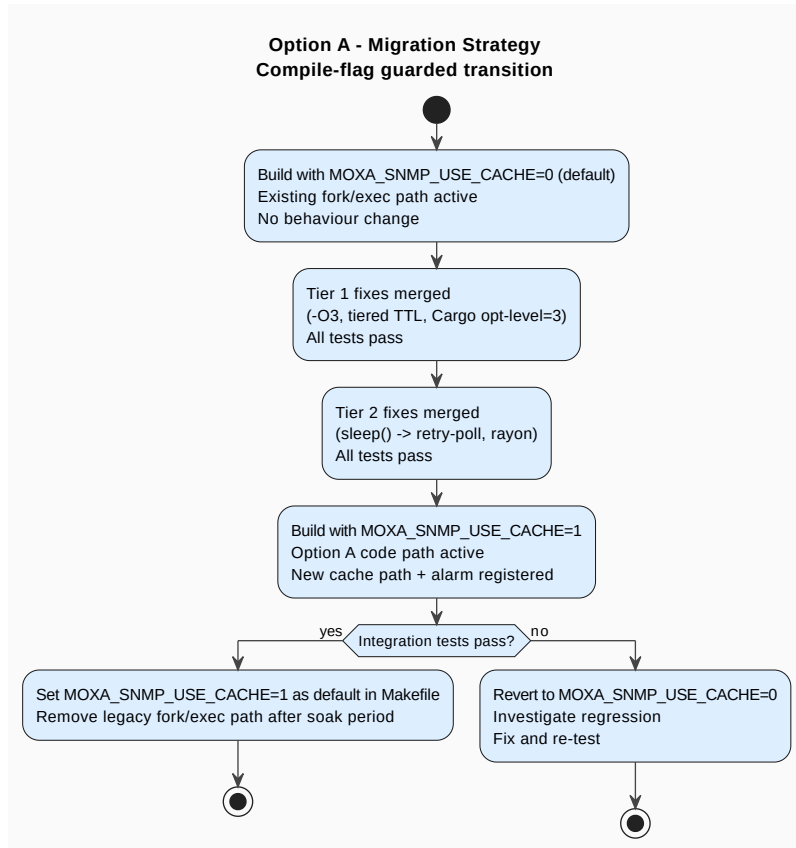
} ies_cache_t;

/* Global instance - initialised in init_ies_auto_mibs() */
extern ies_cache_t g_ies_cache;

/* API */
ies_handle_t ies_cache_init(void);
ies_handle_t ies_cache_get(const char *uri, const char **out_data, size_t *out_len);
ies_handle_t ies_cache_put(const char *uri, const char *data, size_t len);
ies_handle_t ies_cache_invalidate(const char *uri);
void        ies_auto_mibs_cache_refresh(unsigned int clientreg, void *clientarg);

```

8.4 Migration Strategy



Option A: compile-flag guard in ies_auto_mibs.c

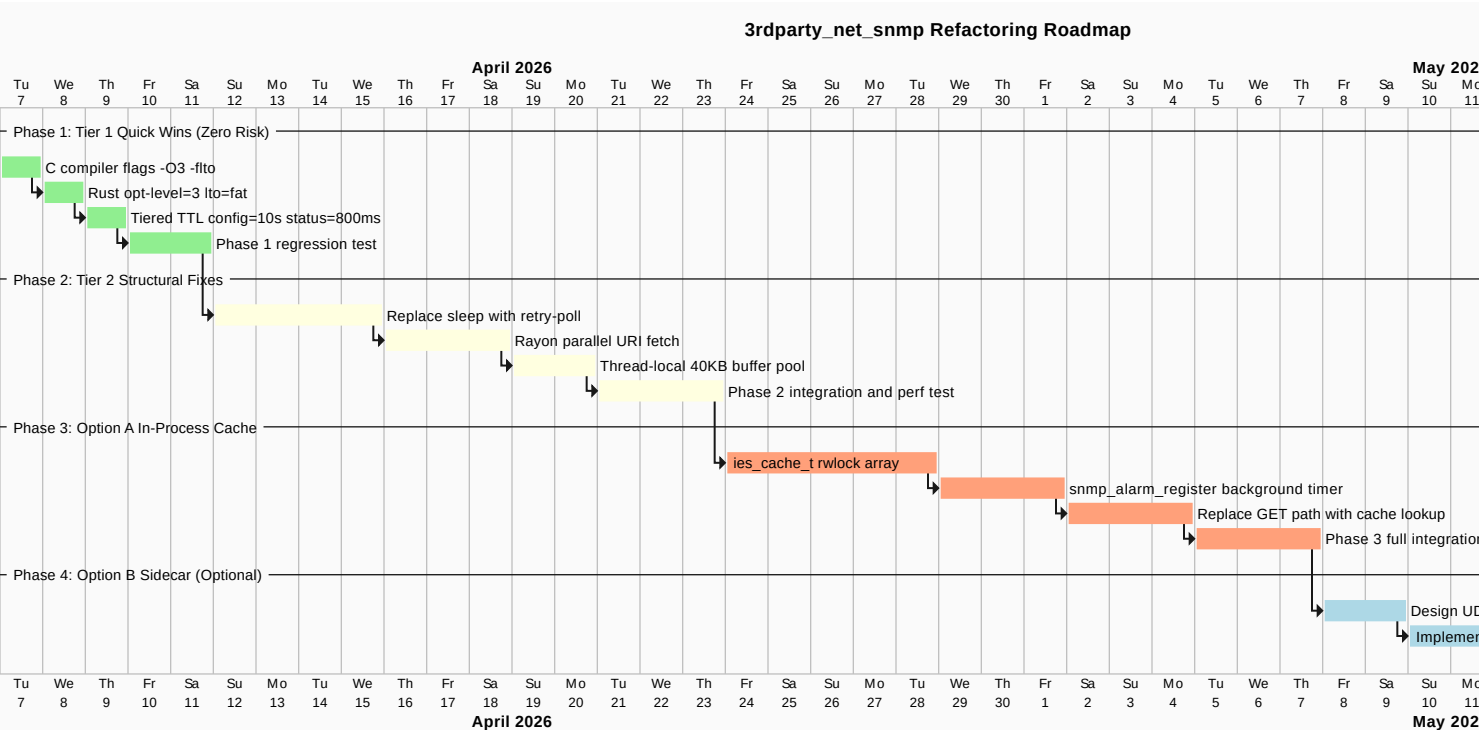
```

/* Wrap new cache path with compile-time flag */
#ifdef MOXA_SNMP_USE_CACHE
#define MOXA_SNMP_USE_CACHE 0 /* default: disabled until validated */
#endif

ies_handle_t entry_handle_check_or_refresh(struct ies_auto_mibs_entry *entry,
                                          char *fname)
{
    #if MOXA_SNMP_USE_CACHE
        /* New path: in-memory cache lookup */
        const char *data = NULL;
        size_t len = 0;
        if (IES_NOERR == ies_cache_get(entry->uri, &data, &len)) {
            return ies_cache_write_to_file(fname, data, len); /* compatibility shim */
        }
        /* Cache miss: trigger background refresh, fall through to legacy path */
    #endif
    /* Legacy path: fork/exec */
    return entry_handle_generate_iss_value_file(entry, NULL);
}

```

9. Implementation Roadmap



10. Testing & Validation

Establish Baseline First

Before making any changes, collect baseline measurements on the target hardware (ARM Cortex-A9 device). Record GET and SET latency, CPU usage under sustained snmpwalk, and file I/O counts. All performance claims must be validated against this baseline on real hardware, not simulation.

Baseline & Validation Measurement Commands

Measurement commands for baseline and validation

```
# --- GET latency measurement (cold cache) ---
# Clear cache files first
rm -f /moxa/run/auto_mibs/*
# Time a single OID GET (repeat 10 times for average)
for i in $(seq 1 10); do
    time snmpget -v2c -c public localhost 1.3.6.1.4.1.8691.10.37.1.0
done

# --- SET latency measurement ---
time snmpset -v2c -c private localhost |
1.3.6.1.4.1.8691.10.37.1.0 i 1

# --- Table walk latency (cold cache) ---
rm -f /moxa/run/auto_mibs/*
time snmpwalk -v2c -c public localhost 1.3.6.1.4.1.8691.10.37

# --- CPU usage under sustained load ---
snmpwalk -v2c -c public localhost 1.3.6.1.4.1.8691 &
pidstat -p $(pgrep snmpd) 1 30

# --- File I/O counter (strace) ---
strace -e trace=open,openat,read,write,rename,stat |
-p $(pgrep snmpd) -f 2>&1 | grep auto_mibs | wc -l

# --- Fork/exec counter ---
strace -e trace=clone,execve -p $(pgrep snmpd) -f 2>&1 | \
grep -E "generate_uri|generate_iss|mv " | wc -l

# --- snmpd memory usage ---
cat /proc/$(pgrep snmpd)/status | grep VmRSS
```

Acceptance Criteria

Test	Baseline (Current)	Tier 1+2 Target	Option A Target
Single OID GET latency (cold)	< 150 ms	< 80 ms	< 5 ms
Single OID GET latency (warm)	< 150 ms	< 80 ms	< 1 ms
SNMP SET latency	< 5000 ms	< 300 ms	< 300 ms

Table walk (32 OIDs)	< 5000 ms	< 1000 ms	< 100 ms
snmpd CPU under snmpwalk	< 80%	< 50%	< 20%
Fork/exec per table walk	> 10	> 10	0
File I/O ops per GET (cold)	> 5	> 3	0
snmpd RSS increase (Option A)	N/A	N/A	< 20 MB
Concurrent GET + SET stability	May timeout	Must pass	Must pass
24h soak test (snmpwalk loop)	Pass	Pass	Pass

11. Risk Register

Risk	Likelihood	Impact	Mitigation
Option A rwlock deadlock (alarm fires during GET holding rdlock)	Low	Critical	Alarm callback uses trylock; falls back to next cycle if lock unavailable
Tiered TTL causes stale config data visible via SNMP	Low	Medium	Config URI TTL = 10s acceptable; SET always invalidates; add ACTIONTYPEINVALIDATE
Rust cdylib ABI incompatibility on ARM	Low	High	Test cdylib linkage on target hardware early; maintain fallback fork/exec path behind compile flag
retry-poll loop busy-waits if ISS is slow	Medium	Medium	Cap poll retries (20x5ms = 100ms max); log and return error if exceeded
rayon thread pool contention with snmpd	Low	Medium	Limit rayon thread count to 2 on ARM (RAYONNUMTHREADS=2); profile under load
40KB thread-local buffer leaked via dlopen unload	Low	Low	Ensure pthreadkeydelete in cdylib destructor
ISS event subscription unreliable (Option C)	High	High	Defer Option C; use TTL-based cache for Option A/B as safety net
snmpd singlethreaded: alarm refresh blocks GET	Medium	Medium	Keep alarm refresh fast (<5ms); move heavy fetch to background pthread for Option A v2
File cache compatibility: existing tools expect files in /moxa/run/auto_mibs/	Medium	Medium	Option A compatibility shim writes to file after cache hit; remove after soak
ARM Cortex-A9 rayon performance regression (threading overhead > gain)	Low	Medium	Benchmark: if N_uris <= 3, sequential is faster; add threshold check

12. Key File Reference

File	Path	Role
iesautomibs.c	net-snmp-5.8/agent/mibgroup/ies-auto-mibs/iesautomibs.c	Main MIB handler: GET/SET dispatch, fork/exec, file cache management
moxasnmphandle_util.c	net-snmp-5.8/agent/mibgroup/ies-auto-mibs/moxasnmphandle_util.c	SET utility: URI config/status/command, blocking sleep() calls
moxasnmptest.h	net-snmp-5.8/agent/mibgroup/ies-auto-mibs/moxasnmptest.h	Central constants: MOXASNMPMAXSTRVALUELEN, MOXASNMPMAXTABLEENTRYTIMEOUT, URI path macros
moxasnmphandle_event.c	net-snmp-5.8/agent/mibgroup/ies-auto-mibs/moxasnmphandle_event.c	Event handler thread: UDS socket server, ACTIONTYPESAVE/EXIT, SOCK_PATH
iesautomibsetup*.c	net-snmp-5.8/agent/mibgroup/ies-auto-mibs/iesautomibsetup*.c	90+ subsystem DB handlers: port, VLAN, RSTP, PoE, QoS, LLDP, etc.
Makefile (top-level)	3rdpartynetsnmp/Makefile	Top-level build: TARGET=armv7-unknown-linux-gnueabi, PATCHLDPATH, snmptools sub-make
Makefile (snmpscritrust)	3rdpartynetsnmp/snmpscritrust/Makefile	Rust build: cargo build --release --target armv7
Cargo.toml (generateurisnmp_table)	snmpscritrust/generateurisnmp_table/Cargo.toml	Rust binary config: opt-level="s" (change to 3), libmoxarust_iss dependency
Cargo.toml (generateisssnmpmergetable)	snmpscritrust/generateisssnmpmergetable/Cargo.toml	Rust binary config for ISS merge table
generateurisnmp_table/src/	snmpscritrust/generateurisnmp_table/src/	Rust source: URI fetch + serialise; add rayon for parallel fetch
generateisssnmpmergetable/src/	snmpscritrust/generateisssnmpmergetable/src/	Rust source: ISS remap merge table generation
iesautomibs_handle.h	snmptools/inc/iesautomibhandle.h	Shared header: struct snmptablesnapshot, struct iesautomibsentry, ieshandle_t

No labels

